

Peticiones HTTP desde nodejs

UD 4. Programación concurrente con NodeJS

Programació de Serveis
i Processos

CFGSG Desenvolupament
d'Aplicacions Multiplataforma



IES Jaume II El Just
Tavernes de la Valldigna

Departament d'Informàtica
Curs 2022-23

Contenidos

1	Introducción. Comunicación entre aplicaciones.	3
2	Peticiones HTTP	3
2.1	Concatenando peticiones	6
3	Peticiones web con fetch	7
4	async/await	11

1 Introducción. Comunicación entre aplicaciones.

Las Apis RESTful (*Representational State Transfer* o Transferencia de Estado Representacional) son una arquitectura de software orientada a recursos (ROA) para implementar *web services*. Se trata de una arquitectura moderna y cada vez más popular, que hace uso del protocolo HTTP para transferencia de datos, y generalmente el formato JSON para su representación. Así pues, con REST podremos crear aplicaciones y servicios entre varios dispositivos, siempre y cuando estas entiendan el protocolo HTTP, siendo Internet el medio natural a través del cual ofrecer estos.

Cómo hemos dicho, REST presenta una arquitectura orientada a recursos, donde cada recurso del servidor dispone de una ruta (dirección URL) diferente, conocida también como *endpoint*. Estos recursos se procesan con los métodos de la interfaz uniforme GET, POST, PUT y DELETE (HTTP tiene más métodos, pero no pertenecen a esta interfaz).

El API que vamos a utilizar para los ejemplos se encuentra documentada en el siguiente repositorio de Github. Para acceder a los recursos, podemos probar con el navegador web las direcciones que en ella se especifican, y que se encuentran alojadas al servicio Railway App.

Por ejemplo, puedes probar desde tu navegador la siguiente dirección URL:

- <https://node-comarques-rest-server-production.up.railway.app/api/provincies>



Cuando hablamos de web services debemos mencionar el protocolo SOAP. Dicho protocolo, fue diseñado antes que REST, y definía la forma en que diferentes plataformas y lenguajes podían intercambiar datos de un modo sencillo, haciendo uso de XML para el intercambio de información.

Más adelante y en otros módulos profundizaremos en este tipo de servicios.

2 Peticiones HTTP

Para poder realizar peticiones web, ya sea a servicios REST o cualquier otro recurso que se encuentre en la web existen diferentes librerías que podemos usar en nuestras aplicaciones javascript.

Cuando trabajamos desde Javascript para el navegador, éste incorpora el objeto `XMLHttpRequest`, que supone la base sobre la que se construyen aplicaciones AJAX (*Asynchronous Javascript XML*). Por su parte, nodejs incorpora de manera estándar los módulos `http` y `https`, que nos permiten tanto crear servidores HTTP/HTTPS como realizar peticiones de este tipo.

El esquema básico de una petición de tipo GET para obtener un recurso web sería el siguiente:

```
const https = require('https');

https.get(URL, (res) => {
  // Variable donde guardar los datos recibidos
  let data = '';

  // Evento que se dispara cada vez que se reciben
  // fragmentos de datos.
  res.on('data', (chunk) => {
    data += chunk;
  });

  // Evento que se dispara cuando se ha recibido
  // toda la respuesta.
  res.on('end', () => {
    console.log(JSON.parse(data).explanation);
  });

  // Evento que se dispara en caso de error.
}).on("error", (err) => {
  console.log("Error: " + err.message);
});
```

La mayoría de funcionalidades del módulo HTTP y HTTPS son de bastante bajo nivel. En este caso, recibimos la respuesta a nuestras peticiones en fragmentos de datos que debemos analizar y reconstruir. Estos fragmentos son bastante sencillos de reconstruir cuando se trata de respuestas en formato JSON, pero no deja de suponer un paso adicional.

Analicemos por partes qué hemos hecho:

- En primer lugar, importamos el objeto https del módulo con el mismo nombre:

```
const https = require('https');
```

- Con este objeto, invocamos a su método get con el siguiente código:

```
https.get(URL, (res) => {
  // Cuerpo del callback

}).on("error", (err) => {
  console.log("Error: " + err.message);
});
```

Como vemos, al método *GET* le proporcionamos la URL o recurso al que deseamos acceder, así como una función de callback, que recibe el parámetro *res* con la respuesta. Si se produce alguna excepción en esta llamada, se ejecuta el segundo fragmento de código, mostrando el error por pantalla.

- Dentro del cuerpo del callback que procesa las respuestas, lo que hacemos es preparar una estructura para almacenar datos cuando son recibidos:

```
let data = '';

res.on('data', (chunk) => {
  data += chunk;
});
```

- Y cuando terminamos de leer datos, lo que hacemos es ya procesar la respuesta completa, en este caso, mostrarla por pantalla:

```
res.on('end', () => {
  console.log(JSON.parse(data).explanation);
});
```

Veamos cómo aplicaríamos este esquema para obtener los datos sobre provincias:

```
const https = require('https');

https.get('https://node-comarques-rest-server-
  ↪ production.up.railway.app/api/provincies', res =>
  ↪ {
    let data = '';

    // Obtenemos el código de estado de la respuesta (200=Ok)
    console.log('Status Code:', res.statusCode);
    // Y la fecha de ésta
    const headerDate = res.headers && res.headers.date ? res.headers.date :
      ↪ 'no response date';
    console.log('Date in Response header:', headerDate);

    res.on('data', chunk => { data += chunk; });

    res.on('end', () => { console.log(JSON.parse(data)); });
  });
}).on('error', err => {
```

```
    console.log('Error: ', err.message);  
  });
```

2.1 Concatenando peticiones

Ahora bien.. si ahora quisiéramos para cada provincia obtener el nombre de las diferentes comarcas, qué haríamos?

```
const https = require('https');  
  
https.get('https://node-comarques-rest-server-  
  ↪ production.up.railway.app/api/provincies', res =>  
  ↪ {  
    let data = '';  
    res.on('data', chunk => {  
      data += chunk;  
    });  
    res.on('end', () => {  
  
      let provinces = JSON.parse(data);  
  
      for (let provincia of provinces) {  
        https.get('https://node-comarques-rest-server-  
  ↪ production.up.railway.app/api/comarques/' + provincia, res =>  
  ↪ {  
          let data = '';  
          res.on('data', chunk => { data += chunk; });  
  
          res.on('end', () => {  
            let comarques = JSON.parse(data);  
  
            console.log(" \nComarques de " + provincia + "\n");  
            for (comarca of comarques) {  
              console.log(comarca.comarca);  
            }  
          });  
        }).on('error', err => {  
          console.log('Error: ', err.message);  
        });  
      }  
    });  
  });
```

```
    }  
  });  
}).on('error', err => {  
  console.log('Error: ', err.message);  
});
```

Como vemos, cuando se finaliza la recepción de las provincias, debemos recorrer éstas, y realizar de nuevo peticiones, que se procesan del mismo modo, añadiendo un nuevo nivel de anidamiento de callbacks.



Disponéis de más información sobre transacciones HTTP en los siguientes enlaces:

- <https://nodejs.org/api/http.html>
- <https://nodejs.org/en/docs/guides/anatomy-of-an-http-transaction/>

3 Peticiones web con fetch

El API *fetch* se introdujo en la versión 6 de EcmaScript, y permite realizar peticiones asíncronas mediante *Promesas*, que también se introdujeron en esta versión. Mediante promesas vamos a poder trabajar con peticiones asíncronas de una manera más cómoda que con los callbacks.

Esta funcionalidad se implementó de forma experimental en el *core* de nodejs en la versión 17.5. Hasta ese momento, se requería de librerías como *Axios* o *node-fetch*.

La forma de realizar una petición con *fetch* es relativamente sencilla, y consiste en invocar a *fetch* proporcionándole como argumento la URL del recurso que deseamos obtener. Por ejemplo:

```
const res = fetch('https://node-comarques-rest-server-  
production.up.railway.app/api/provincies');
```

El API *fetch* devuelve un objeto de tipo *Promise* con sus dos métodos *then()* y *catch()*. Estos métodos reciben sendas funciones de *callback* que serán invocadas bien cuando se obtenga respuesta o bien cuando se produzca un error. Hay que tener en cuenta que los errores que harán que se ejecute el *catch* son aquellos relacionados con errores de red o de otro tipo, pero no los errores HTTP que devuelven un código de error (tipo 404, etc). Estos errores se tendrán que tratar dentro del *then()*.

```
res.then(function(result) {  
    // Procesamiento de la respuesta  
})  
.catch(function(err) {  
    // Tratamiento de errores de red  
});
```

O usando funciones flecha:

```
res.then((result) => {  
    // Procesamiento de la respuesta  
})  
.catch((err) => {  
    // Tratamiento de errores de red  
});
```

También hay que tener en cuenta que lo más habitual será encontrarnos con dos promesas encadenadas para acceder al cuerpo del mensaje que nos devuelve el servidor. Veámoslo con un ejemplo. Para el caso de obtener las provincias, si mostramos el resultado por pantalla:

```
const res = fetch('https://node-comarques-rest-server-  
    ↪ production.up.railway.app/api/provincies');  
  
res.then((result) => {  
    console.log(result);  
});
```

Nos imprimirá el JSON con todo el objeto Response devuelto por el servidor:

```
Response {  
    ...  
    [Symbol(state)]: {  
        aborted: false,  
        rangeRequested: false,  
        timingAllowPassed: true,  
        requestIncludesCredentials: true,  
        type: 'default',  
        status: 200,  
        timingInfo: {
```



```

    startTime: 75.08328900486231,
    ...
  },
  cacheState: '',
  statusText: 'OK',
  headersList: HeadersList {...},
  urlList: [ [URL] ],
  body: { stream: undefined }
},
[Symbol(headers)]: HeadersList {
  [Symbol(headers map)]: Map(9) {
    'x-powered-by' => 'Express',
    'access-control-allow-origin' => '*',
    'access-control-allow-headers' => 'Origin, X-Requested-With,
    ↪ Content-Type, Accept',
    'content-type' => 'application/json; charset=utf-8',
    'date' => 'Mon, 19 Dec 2022 17:32:27 GMT',
    'server' => 'railway'
    ...
  },...}
}

```

Este objeto de tipo `Response` que se nos devuelve, no dispone del contenido del cuerpo (*body*) de forma directa, sino que proporciona diferentes métodos para acceder al contenido, como son `.json()` para obtener el contenido en formato JSON o bien `.text()`, para obtenerlo en formato texto. Estos métodos `.json()` y `.text()`, nos devuelven otra promesa, que se resolverá cuando se haya obtenido el contenido en uno u otro formato. Esta promesa será tratada con otro `fetch` encadenado, por ejemplo, para mostrar el resultado:

```

const res = fetch('https://node-comarques-rest-server-
  ↪ production.up.railway.app/api/provincies');

res.then((result) => {
  // Obtenemos el contenido en JSON
  return result.json();
}).then((result) => {
  // Mostramos el JSON por pantalla
  console.log(result);
});

```

Aparte de `text` y `json`, el objeto *Response* proporciona otros métodos para el acceso al contenido.

Veámoslos en la siguiente tabla:

Método	Significado	Ejemplo
<code>response.text()</code>	Retorn el contenido en formado texto	<code>["València","Alacant","Castelló"]</code>
<code>response.json()</code>	Retorna el contenido en formato JSON	<code>['València', 'Alacant', 'Castelló']</code>
<code>response.arrayBuffer()</code>	Retorna el contenido en formato ArrayBuffer	<code>ArrayBuffer { byteLength: 35 }</code>
<code>response.blob()</code>	Retorna el contenido en formato Blob para gestionarlo con <code>URL.createObjectURL()</code>	<code>data = Blob { [Symbol(type)]: 'application/json; charset=utf-8', [Symbol(buffer)]: <Buffer 5b 22 56 61 6c c3 a8 6e 63 69 61 22 2c 22 41 6c 61 63 61 6e 74 22 2c 22 43 61 73 74 65 6c 6c c3 b3 22 5d> }</code> <code>URL.createObjectURL(data)=blob:null/d7cf510254-4feb-bdbe-cf9aaf1863ac</code>
<code>response.formData()</code>	Retorna el contenido en formato FormData	

Hay que tener en cuenta que estos métodos nos devuelven realmente una promesa, y cuando esta se resuelve, su método `then()` nos pasará el resultado a la función que le añadimos como callback. De este modo, solo podemos obtener el cuerpo de la respuesta una vez, tras la cual, ya no podremos volver a solicitar otra conversión. Para resolver esta situación, el objeto *Response* dispone del método `clone()`, que nos permite duplicar un objeto y gestionar varias veces el cuerpo (*body*). Vemos-lo con un ejemplo:

```
fetch('https://node-comarques-rest-server-
  ↪ production.up.railway.app/api/provincies')
  .then(function(response) {

    let resposta1 = response.clone();
    // Clonamos la respuesta (response)
    // como resposta1 y la consumimos como texto.
```

```
    resposta1.text().then(function(data) {  
        console.log(data);  
    });  
  
    let resposta2 = response.clone();  
    // Volvemos a clonar response, ahora  
    // como resposta2 y la consumimos como ko  
    resposta2.json().then(function(data) {  
        console.log(data);  
    });  
  
    // Ahora clonamos en resposta3 y  
    // la tratamos como un arrayBuffer.  
    let resposta3 = response.clone();  
    resposta3.arrayBuffer().then(function(data) {  
        console.log(data);  
    });  
  
    // Y finalmente, resposta4 como blob.  
    let resposta4 = response.clone();  
    resposta4.blob().then(function(data) {  
        console.log(data);  
        console.log(URL.createObjectURL(data));  
    });  
  
    })  
    .catch(function(err) {  
        console.error(err);  
    });  
});
```

Con esto, hemos obtenido el resultado en diferentes formatos haciendo solo uso de un `fetch` y clonando la respuesta para obtener esta en diferentes formatos. Luego, utilizamos los diferentes métodos que nos devuelven promesas, y tratamos estas dentro del mismo `then` que nos devuelve el `fetch` (como podéis ver, ahora no hacemos el `return`).

4 `async/await`

En el apartado anterior ya introdujimos la mejora sintáctica `async/await` de EcmaScript 2017. Esta misma sintaxis podemos utilizarla para procesar las promesas que recibimos cuando usamos la API `Fetch`.

El ejemplo de obtención de las provincias sería:

```
async function obtenerProvincias() {  
  let respuesta = await fetch('https://node-comarques-rest-server-  
    ↪ production.up.railway.app/api/provincies');  
  let respuestaJSON = await respuesta.json();  
  console.log(respuestaJSON);  
}  
  
obtenerProvincias();
```